



Small. Fast. Reliable.
Choose any three.

- [Home](#)
 - [Menu](#)
 - [About](#)
 - [Documentation](#)
 - [Download](#)
 - [License](#)
 - [Support](#)
 - [Purchase](#)
-
- [About](#)
 - [Documentation](#)
 - [Download](#)
 - [Support](#)
 - [Purchase](#)

Search Documentation ▼

SQLite As An Application File Format

► Table Of Contents

1. Executive Summary

An SQLite database file with a defined schema often makes an excellent application file format. Here are a dozen reasons why this is so:

1. Simplified Application Development
2. Single-File Documents
3. High-Level Query Language
4. Accessible Content
5. Cross-Platform
6. Atomic Transactions
7. Incremental And Continuous Updates
8. Easily Extensible
9. Performance
10. Concurrent Use By Multiple Processes
11. Multiple Programming Languages
12. Better Applications

Each of these points will be described in more detail below, after first considering more closely the meaning of "application file format". See also the [short version](#) of this whitepaper.

2. What Is An Application File Format?

An "application file format" is the file format used to persist application state to disk or to exchange information between programs. There are thousands of application file formats in use today. Here are just a few examples:

- DOC - Word Perfect and Microsoft Office documents
- DXF - AutoCAD drawings

- PDF - Portable Document Format from Adobe
- XLS - Microsoft Excel Spreadsheet
- GIT - Git source code repository
- EPUB - The Electronic Publication format used by non-Kindle eBooks
- ODT - The Open Document format used by OpenOffice and others
- PPT - Microsoft PowerPoint presentations
- ODP - The Open Document presentation format used by OpenOffice and others

We make a distinction between a "file format" and an "application format". A file format is used to store a single object. So, for example, a GIF or JPEG file stores a single image, and an XHTML file stores text, so those are "file formats" and not "application formats". An EPUB file, in contrast, stores both text and images (as contained XHTML and GIF/JPEG files) and so it is considered an "application format". This article is about "application formats".

The boundary between a file format and an application format is fuzzy. This article calls JPEG a file format, but for an image editor, JPEG might be considered the application format. Much depends on context. For this article, let us say that a file format stores a single object and an application format stores many different objects and their relationships to one another.

Most application formats fit into one of these three categories:

1. **Fully Custom Formats.** Custom formats are specifically designed for a single application. DOC, DXF, PDF, XLS, and PPT are examples of custom formats. Custom formats are usually contained within a single file, for ease of transport. They are also usually binary, though the DXF format is a notable exception. Custom file formats require specialized application code to read and write and are not normally accessible from commonly available tools such as unix command-line programs and text editors. In other words, custom formats are usually "opaque blobs". To access the content of a custom application file format, one needs a tool specifically engineered to read and/or write that format.
2. **Pile-of-Files Formats.** Sometimes the application state is stored as a hierarchy of files. Git is a prime example of this, though the phenomenon occurs frequently in one-off and bespoke applications. A pile-of-files format essentially uses the filesystem as a key/value database, storing small chunks of information into separate files. This gives the advantage of making the content more accessible to common utility programs such as text editors or "awk" or "grep". But even if many of the files in a pile-of-files format are easily readable, there are usually some files that have their own custom format (example: Git "Packfiles") and are hence "opaque blobs" that are not readable or writable without specialized tools. It is also much less convenient to move a pile-of-files from one place or machine to another, than it is to move a single file. And it is hard to make a pile-of-files document into an email attachment, for example. Finally, a pile-of-files format breaks the "document metaphor": there is no one file that a user can point to that is "the document".
3. **Wrapped Pile-of-Files Formats.** Some applications use a Pile-of-Files that is then encapsulated into some kind of single-file container, usually a ZIP archive. EPUB, ODT, and ODP are examples of this approach. An EPUB book is really just a ZIP archive that contains various XHTML files for the text of book chapters, GIF and JPEG images for the artwork, and a specialized catalog file that tells the eBook reader how all the XML and image files fit together. OpenOffice documents (ODT and ODP) are also ZIP archives containing XML and images that represent their content as well as "catalog" files that show the interrelationships between the component parts.

A wrapped pile-of-files format is a compromise between a full custom file format and a pure pile-of-files format. A wrapped pile-of-files format is not an opaque blob in the same sense as a custom format, since the component parts can still be accessed using any common ZIP archiver, but the format is not quite as accessible as a pure pile-of-files format because one does still need the ZIP archiver, and one cannot normally use command-line tools like "find" on the file hierarchy without first un-zipping it. On the other

hand, a wrapped pile-of-files format does preserve the document metaphor by putting all content into a single disk file. And because it is compressed, the wrapped pile-of-files format tends to be more compact.

As with custom file formats, and unlike pure pile-of-file formats, a wrapped pile-of-files format is not as easy to edit, since usually the entire file must be rewritten in order to change any component part.

The purpose of this document is to argue in favor of a fourth new category of application file format: An SQLite database file.

3. SQLite As The Application File Format

Any application state that can be recorded in a pile-of-files can also be recorded in an SQLite database with a simple key/value schema like this:

```
CREATE TABLE files(filename TEXT PRIMARY KEY, content BLOB);
```

If the content is compressed, then such an [SQLite Archive](#) database is [the same size](#) ($\pm 1\%$) as an equivalent ZIP archive, and it has the advantage of being able to update individual "files" without rewriting the entire document.

But an SQLite database is not limited to a simple key/value structure like a pile-of-files database. An SQLite database can have dozens or hundreds or thousands of different tables, with dozens or hundreds or thousands of fields per table, each with different datatypes and constraints and particular meanings, all cross-referencing each other, appropriately and automatically indexed for rapid retrieval, and all stored efficiently and compactly in a single disk file. And all of this structure is succinctly documented for humans by the SQL schema.

In other words, an SQLite database can do everything that a pile-of-files or wrapped pile-of-files format can do, plus much more, and with greater lucidity. An SQLite database is a more versatile container than a key/value filesystem or a ZIP archive. (For a detailed example, see the [OpenOffice case study](#) essay.)

The power of an SQLite database could, in theory, be achieved using a custom file format. But any custom file format that is as expressive as a relational database would likely require an enormous design specification and many tens or hundreds of thousands of lines of code to implement. And the end result would be an "opaque blob" that is inaccessible without specialized tools.

Hence, in comparison to other approaches, the use of an SQLite database as an application file format has compelling advantages. Here are a few of these advantages, enumerated and expounded:

1. **Simplified Application Development.** No new code is needed for reading or writing the application file. One has merely to link against the SQLite library, or include the [single "sqlite3.c" source file](#) with the rest of the application C code, and SQLite will take care of all of the application file I/O. This can reduce application code size by many thousands of lines, with corresponding saving in development and maintenance costs.

SQLite is one of the [most used](#) software libraries in the world. There are literally tens of billions of SQLite database files in use daily, on smartphones and gadgets and in desktop applications. SQLite is [carefully tested](#) and proven reliable. It is not a component that needs much tuning or debugging, allowing developers to stay focused on application logic.

2. **Single-File Documents.** An SQLite database is contained in a single file, which is easily copied or moved or attached. The "document" metaphor is preserved.

SQLite does not have any file naming requirements and so the application can use any custom file suffix that it wants to help identify the file as "belonging" to the application. SQLite database files contain a 4-byte [Application ID](#) in their headers that can be set to an application-defined value and then used to

identify the "type" of the document for utility programs such as [file\(1\)](#), further enhancing the document metaphor.

3. **High-Level Query Language.** SQLite is a complete relational database engine, which means that the application can access content using high-level queries. Application developers need not spend time thinking about "how" to retrieve the information they need from a document. Developers can write SQL that expresses "what" information they want and let the database engine figure out how to best retrieve that content. This helps developers operate "heads up" and remain focused on solving the user's problem, and avoid time spent "heads down" fiddling with low-level file formatting details.

A pile-of-files format can be viewed as a key/value database. A key/value database is better than no database at all. But without transactions or indices or a high-level query language or a proper schema, it is much harder and more error prone to use a key/value database than a relational database.

4. **Accessible Content.** Information held in an SQLite database file is accessible using commonly available open-source command-line tools - tools that are installed by default on Mac and Linux systems and that are freely available as a self-contained EXE file on Windows. Unlike custom file formats, application-specific programs are not required to read or write content in an SQLite database. An SQLite database file is not an opaque blob. It is true that command-line tools such as text editors or "grep" or "awk" are not useful on an SQLite database, but the SQL query language is a much more powerful and convenient way for examining the content, so the inability to use "grep" and "awk" and the like is not seen as a loss.

An SQLite database is a [well-defined and well-documented](#) file format that is in widespread use by literally millions of applications and is backwards compatible to its inception in 2004 and which promises to continue to be compatible in decades to come. The longevity of SQLite database files is particularly important to bespoke applications, since it allows the document content to be accessed far in the future, long after all traces of the original application have been lost. Data lives longer than code. SQLite databases are [recommended by the US Library of Congress](#) as a storage format for long-term preservation of digital content.

5. **Cross-Platform.** SQLite database files are portable between 32-bit and 64-bit machines and between big-endian and little-endian architectures and between any of the various flavors of Windows and Unix-like operating systems. The application using an SQLite application file format can store binary numeric data without having to worry about the byte-order of integers or floating point numbers. Text content can be read or written as UTF-8, UTF-16LE, or UTF-16BE and SQLite will automatically perform any necessary translations on-the-fly.

6. **Atomic Transactions.** Writes to an SQLite database are [atomic](#). They either happen completely or not at all, even during system crashes or power failures. So there is no danger of corrupting a document just because the power happened to go out at the same instant that a change was being written to disk.

SQLite is transactional, meaning that multiple changes can be grouped together such that either all or none of them occur, and so that the changes can be rolled back if a problem is found prior to commit. This allows an application to make a change incrementally, then run various sanity and consistency checks on the resulting data prior to committing the changes to disk. The [Fossil](#) DVCS [uses this technique](#) to verify that no repository history has been lost prior to each change.

7. **Incremental And Continuous Updates.** When writing to an SQLite database file, only those parts of the file that actually change are written out to disk. This makes the writing happen faster and saves wear on SSDs. This is an enormous advantage over custom and wrapped pile-of-files formats, both of which usually require a rewrite of the entire document in order to change a single byte. Pure pile-of-files formats can also do incremental updates to some extent, though the granularity of writes is usually larger with pile-of-file formats (a single file) than with SQLite (a single page).

SQLite also supports continuous update. Instead of collecting changes in memory and then writing them to disk only on a File/Save action, changes can be written back to the disk as they occur. This avoids loss of work on a system crash or power failure. An [automated undo/redo stack](#), managed using triggers, can be kept in the on-disk database, meaning that undo/redo can occur across session boundaries.

8. **Easily Extensible.** As an application grows, new features can be added to an SQLite application file format simply by adding new tables to the schema or by adding new columns to existing tables. Adding columns or tables does not change the meaning of prior queries, so with a modicum of care to ensuring that the meaning of legacy columns and tables are preserved, backwards compatibility is maintained.

It is possible to extend custom or pile-of-files formats too, of course, but doing so is often much harder. If indices are added, then all application code that changes the corresponding tables must be located and modified to keep those indices up-to-date. If columns are added, then all application code that accesses the corresponding table must be located and modified to take into account the new columns.

9. **Performance.** In many cases, an SQLite application file format will be [faster than a pile-of-files format](#) or a custom format. In addition to being faster for raw read and writes, SQLite can often dramatically improve start-up times because instead of having to read and parse the entire document into memory, the application can do queries to extract only the information needed for the initial screen. As the application progresses, it only needs to load as much material as is needed to draw the next screen, and can discard information from prior screens that is no longer in use. This helps keep the memory footprint of the application under control.

A pile-of-files format can be read incrementally just like SQLite. But many developers are surprised to learn that SQLite can read and write smaller BLOBs (less than about 100KB in size) from its database faster than those same blobs can be read or written as separate files from the filesystem. (See [35% Faster Than The Filesystem](#) and [Internal Versus External BLOBs](#) for further information.) There is overhead associated with operating a relational database engine, however one should not assume that direct file I/O is faster than SQLite database I/O, as often it is not.

In either case, if performance problems do arise in an SQLite application those problems can often be resolved by adding one or two [CREATE INDEX](#) statements to the schema or perhaps running [ANALYZE](#) one time and without having to touch a single line of application code. But if a performance problem comes up in a custom or pile-of-files format, the fix will often require extensive changes to application code to add and maintain new indices or to extract information using different algorithms.

10. **Concurrent Use By Multiple Processes.** SQLite automatically coordinates concurrent access to the same document from multiple threads and/or processes. Two or more applications can connect and read from the same document at the same time. Writes are serialized, but as writes normally only take milliseconds, applications simply take turns writing. SQLite automatically ensures that the low-level format of the document is uncorrupted. Accomplishing the same with a custom or pile-of-files format, in contrast, requires extensive support in the application. And the application logic needed to support concurrency is a notorious bug-magnet.
11. **Multiple Programming Languages.** Though SQLite is itself written in ANSI-C, interfaces exist for just about every other programming language you can think of: C++, C#, Objective-C, Java, Tcl, Perl, Python, Ruby, Erlang, JavaScript, and so forth. So programmers can develop in whatever language they are most comfortable with and which best matches the needs of the project.

An SQLite application file format is a great choice in cases where there is a collection or "federation" of separate programs, often written in different languages and by different development teams. This comes up commonly in research or laboratory environments where one team is responsible for data acquisition and other teams are responsible for various stages of analysis. Each team can use whatever hardware, operating system, programming language and development methodology that they are most comfortable with, and as long as all programs use an SQLite database with a common schema, they can all interoperate.

12. **Better Applications.** If the application file format is an SQLite database, the complete documentation for that file format consists of the database schema, with perhaps a few extra words about what each table and column represents. The description of a custom file format, on the other hand, typically runs on for hundreds of pages. A pile-of-files format, while much simpler and easier to describe than a fully custom format, still tends to be much larger and more complex than an SQL schema dump, since the names and format for the individual files must still be described.

This is not a trivial point. A clear, concise, and easy to understand file format is a crucial part of any application design. Fred Brooks, in his all-time best-selling computer science text, *The Mythical Man-Month* says:

Representation is the essence of computer programming.

...

Show me your flowcharts and conceal your tables, and I shall continue to be mystified. Show me your tables, and I won't usually need your flowcharts; they'll be obvious.

Rob Pike, in his *Rules of Programming* expresses the same idea this way:

Data dominates. If you've chosen the right data structures and organized things well, the algorithms will almost always be self-evident. Data structures, not algorithms, are central to programming.

Linus Torvalds used different words to say much the same thing on the Git mailing list on 2006-06-27:

Bad programmers worry about the code. Good programmers worry about data structures and their relationships.

The point is this: an SQL database schema almost always does a far better job of defining and organizing the tables and data structures and their relationships. And having clear, concise, and well-defined representation almost always results in an application that performs better, has fewer problems, and is easier to develop and maintain.

4. Security Considerations

SQLite is robust against maliciously malformed database files and SQL inputs. An attacker will not be able to provoke a memory error by corrupting an SQLite database used as an application file.

There are attacks that a clever attacker can carry out against an application by tricking a user to open an application file that is an SQLite database. However, a few simple precautions can prevent any serious infiltration or damage. If you do choose to use SQLite (or any relational database for that matter) as an application file format, please review the [defense against dark arts](#) document, and especially the section on [untrusted database files](#) to make sure you have prudent defensive measures in place and have not accidentally opened any security holes.

5. Conclusion

SQLite is not the perfect application file format for every situation. But in many cases, SQLite is a far better choice than either a custom file format, a pile-of-files, or a wrapped pile-of-files. SQLite is a high-level, stable, reliable, cross-platform, widely-deployed, extensible, performant, accessible, concurrent file format. It deserves your consideration as the standard file format on your next application design.